Journal of Statistical Software

MMMMMM YYYY, Volume VV, Issue II. *doi: 10.18637/jss.v000.i00*

bioLeak: Leakage-Aware Modeling and Diagnostics for Machine Learning in R

Selçuk Korkmaz
Trakya University

Abstract

Data leakage remains a recurrent source of optimistic bias in biomedical machine learning studies. Standard row-wise cross-validation and globally estimated preprocessing steps are often inappropriate for data with repeated measurements, study-level heterogeneity, batch effects, or temporal dependencies. This paper describes **bioLeak**, an R package for constructing leakage-aware resampling workflows and for auditing fitted models for common leakage mechanisms. The package provides leakage-aware split construction, train-fold-only preprocessing, cross-validated model fitting, nested hyperparameter tuning, post hoc leakage audits, and HTML reporting. The primary functions are `make_split_plan`, `fit_resample`, `tune_resample`, `audit_leakage`, `audit_report`, and `delta_lsi`. The implementation supports binary classification, multiclass classification, continuous regression, and survival analysis, with task-specific metrics and S4 containers for splits, fits, audits, and inflation summaries. The simulation artifacts show how apparent performance changes under controlled leakage mechanisms, and the case study illustrates how guarded and naive pipelines can yield materially different conclusions on multi-study transcriptomic data. The emphasis throughout is on software design, reproducible workflows, and interpretation of diagnostic output.

Keywords: biomedical machine learning, data leakage, cross-validation, reproducibility, resampling, auditing, R.

1. Introduction

Cross-validated performance estimates are often interpreted as evidence that a predictive model will generalize to future biomedical data. That interpretation depends on assumptions that are frequently violated in practice: observations must be exchangeable with respect to the resampling design, preprocessing must be estimated without access to evaluation data, and the outcome must not be recoverable from proxy variables that would be unavailable at prediction

time. Violations of these assumptions are commonly described as data leakage. Leakage is not a minor implementation detail. It affects whether estimated discrimination, calibration, and downstream biological interpretation are credible. This concern has been recognized for decades across statistical learning and biomedical prediction, from early discussions of selection bias in microarray studies (**?**) and broader treatments of leakage in data mining (**?**) to warnings about genomic prediction pitfalls (**??**) and recent accounts linking leakage to wider reproducibility failures in machine-learning-based science (**?**). Closely related evaluation bias also arises when model selection is not nested appropriately within resampling (**??**).

The problem is especially acute in biomedical applications. A single study may include repeated measurements from the same participant, technical replicates, laboratory batches, study-site effects, or longitudinal samples collected over time. In such settings, ordinary row-wise cross-validation can place correlated samples in both training and test folds. Likewise, global imputation, normalization, filtering, or feature screening performed before resampling can transmit information from assessment data into the training pipeline. These choices can materially inflate apparent performance while leaving little trace in a conventional model summary, a practical concern in health-care machine learning where evaluation, implementation, and deployment constraints are already difficult to align (**??**).

Recent methodological work has further clarified the scale and heterogeneity of the problem. **?** described leakage as one contributor to broader reproducibility failures in machine-learning-based science. In neuroimaging, **?** showed that repeated-subject leakage and feature-selection leakage can substantially inflate predictive performance, while **?** reported that leakage-compatible practices were common enough to affect meta-analytic conclusions in psychiatric imaging. Taken together, these studies suggest that leakage cannot be addressed by a single generic rule. It is heterogeneous, domain specific, and tightly tied to evaluation design.

Existing modeling frameworks provide much of the infrastructure needed for modern predictive analysis, but they do not by themselves define valid leakage boundaries. In R (**?**), **caret**, **tidymodels**, and **mlr3** support flexible resampling, preprocessing, and model-fitting workflows (**????**). These frameworks can be used safely, but they rely on the analyst to encode resampling units, grouping variables, temporal structure, and fold-wise preprocessing correctly. In biomedical applications, where rows are often nested within subjects, batches, studies, or time windows, the central difficulty is not only fitting a model but specifying which observations may legitimately share information under the intended prediction setting.

bioLeak was developed to make those leakage boundaries explicit and operational within a single workflow. The package addresses two related tasks. The first is prevention: constructing split plans and preprocessing pipelines that reduce the risk of train–test contamination. The second is diagnosis: auditing fitted resampling results for patterns that are compatible with leakage or other structured sources of spurious signal and that may indicate inflated performance. Concretely, **bioLeak** provides leakage-aware split planning, overlap checking, guarded preprocessing, resampling-based model fitting, tuning where supported, and post hoc audit utilities within a unified R interface. The package is intended for practical use with tabular biomedical data and `SummarizedExperiment` objects, and it interoperates with widely used R infrastructure including **parsnip**, **recipes**, **workflows**, **yardstick**, and **rsample**.

The contribution of **bioLeak** is therefore not a new learner. Rather, it is a software framework for representing leakage-aware evaluation design as a first-class object, enforcing key train–test separation rules during preprocessing and resampling, and attaching audit procedures to

fitted workflows so that suspicious performance inflation can be examined explicitly rather than inferred informally after the fact. This combination is particularly useful in biomedical settings, where dependency structure is often known in principle but inconsistently encoded in routine modeling practice.

The remainder of the manuscript is organized as follows. Section **??** summarizes the leakage mechanisms that motivate the package. Section **??** describes the main software components. Section **??** presents a reproducible workflow based on the package API. Section **??** introduces the `delta_lsi()` framework for quantifying performance inflation between naive and guarded pipelines. Section **??** reports simulation results, Section **??** presents a multi-study transcriptomic case study, and Section **??** discusses software design and implementation. Section **??** addresses limitations and future directions.

2. Data Leakage in Biomedical Machine Learning

2.1. Preprocessing leakage

Preprocessing leakage occurs when quantities used to transform predictors are estimated using information from outside the training fold, rather than being learned strictly within each resampling split. Common examples include missing-value imputation, normalization, feature filtering, and feature selection applied before resampling or estimated on the full dataset. **?** showed that even unsupervised preprocessing can bias cross-validation when transformation parameters are estimated outside the resampling loop. This problem is common in biomedical pipelines, where normalization and predictor filtering are often treated as preliminary data-cleaning steps rather than as components of the modeling procedure. In genomics, the same issue is often described as feature-selection bias: selecting genes before resampling can produce substantially inflated performance estimates even when the downstream classifier itself is simple (**???**).

2.2. Dependent samples and subject overlap

Many biomedical datasets contain repeated measures, multi-region samples from one patient, family-related observations, or longitudinal observations from the same individual. If row-wise resampling is applied, records from the same underlying unit can appear in both training and test folds. In that case, the model may partially recognize subject identity or other unit-specific structure rather than the biological signal of interest. This risk is especially high when outcomes are stable within individuals or when repeated measurements are closely spaced and therefore highly correlated. More generally, this problem belongs to the broader class of structured resampling failures discussed by **?**, **?**, and **?**, in which naive row-wise fold assignment ignores dependencies induced by grouping, time, space, or hierarchy.

2.3. Batch and study confounding

High-throughput molecular assays frequently exhibit batch effects associated with laboratory, plate, scanner, platform, or study site. If outcome labels are unevenly distributed across these technical groups, a model can use batch as a surrogate for outcome. **?** and **?** showed that such confounding, as well as attempts to remove it improperly, can bias cross-validation-based

performance estimates. In multi-study settings, however, the intended validation target is critical: if one study is held out at a time for external validation, perfect fold–study association is expected and should not be interpreted as a software error. More generally, valid resampling must respect the dependence structure that is relevant to the intended deployment setting (**?**).

2.4. Temporal look-ahead leakage

When data have a temporal order, random splitting can allow the training set to use information that would not have been available at the time predictions are made for the test set. The most direct example is an explicit future variable, such as a subsequent laboratory measurement or a later event summary, included as a predictor. A subtler form arises when preprocessing or feature construction uses information estimated from future observations. Proper time-aware resampling therefore requires chronological ordering and, in some settings, additional separation between training and evaluation periods to prevent temporal overlap. More generally, evaluation is meaningful only when the data split matches the prediction task that the model is intended to support (**???**).

2.5. Target leakage and duplicate samples

Target leakage occurs when a predictor carries direct information about the outcome definition. In electronic health records, this may take the form of a billing code, procedure, or medication that is downstream of the diagnosis. In molecular or public expression datasets, it may arise through label-derived features, transformed outcome summaries, or variables created using information unavailable at prediction time. Duplicate and near-duplicate samples are related but distinct: they do not need to encode the outcome explicitly, yet a nearly identical training sample can make a test sample artificially easy to predict. Because these problems can persist even when resampling appears well designed, they are natural targets for post hoc diagnostics. Near-duplicate detection inherits standard design choices from record linkage and similarity search—feature representation, matching rules, and similarity thresholds—rather than classical hypothesis testing (**???**).

3. Overview of the bioLeak Package

The package is organized around four design goals: leakage-aware modeling, reproducible workflows, diagnostic auditing, and compatibility with common R modeling tools. The goal is to evaluate whether reported performance may be inflated by leakage or other structured sources of spurious signal.

Figure **??** summarizes the main package workflow from a labeled dataset to fitted resampling results, audit summaries, and inflation quantification.

Each stage produces an S4 object: `LeakSplits`, `LeakFit`, `LeakAudit`, and `LeakDeltaLSI`. These objects store the intermediate state needed for most downstream analysis and reporting steps.

The main workflow begins with `make_split_plan()`, which constructs an S4 `LeakSplits` object. The split plan records the resampling mode, fold-level indices or compact fold assignments, and metadata such as the grouping variables, number of folds, number of repeats,

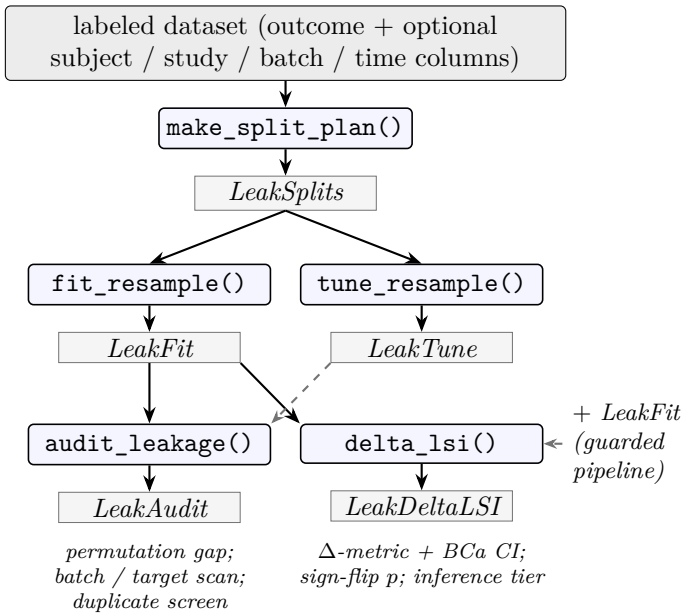

Figure 1: **bioLeak** leakage-aware modeling workflow. Rounded boxes (`monospace`) are function calls; italic labels are the objects they return. Solid arrows show the primary data flow; dashed arrows indicate secondary inputs. `audit_leakage()` accepts both `LeakFit` and `LeakTune` objects; `delta_lsi()` compares two `LeakFit` objects (one from each pipeline).

and a reproducibility hash. The split modes most relevant to the present manuscript are `subject_grouped`, `batch_blocked`, `study_loocv`, and `time_series`. These modes address distinct dependence structures rather than acting as interchangeable cross-validation variants.

Model fitting is handled by `fit_resample()`, which applies guarded preprocessing independently within each fold, trains one or more learners, and returns a `LeakFit` object. The fit object stores per-fold metrics, aggregated summaries, audit metadata, predictions, preprocessing state, learners, and information required for later refit-based audits. The function accepts data frames, matrices, and `SummarizedExperiment` objects (the package vignette demonstrates `SummarizedExperiment` input). Learners may be specified through **parsnip**, **workflows**, legacy character interfaces, or custom fit/predict pairs.

`tune_resample()` extends this design to nested resampling. Outer folds are defined by a `LeakSplits` object or an **rsample** structure that already contains inner folds. Tuning results are returned as a `LeakTune` object containing inner tuning outputs, outer-fold metrics, selected hyperparameters, optional threshold selections for binary outcomes, and an optional refitted final workflow. Survival tasks are currently excluded from `tune_resample()`.

Auditing is provided by `audit_leakage()`, which returns a `LeakAudit` object. The audit combines a permutation-gap analysis, fold association tests for batch or study metadata, target-leakage scans on a reference feature matrix, duplicate detection, and a compact mechanism-level summary stored in the object metadata. `audit_report()` renders these results to a self-contained HTML report based on a bundled R Markdown template. For direct inflation comparisons between a naive and a guarded pipeline, the package also provides `delta_lsi()`, which returns a `LeakDeltaLSI` object summarizing repeat-level performance differences, ro-

Table 1: Primary software components in **bioLeak**.

Stage	**Primary function(s)**	**Main output or role**
Split construction	`make_split_plan()`	Leakage-aware split plans for subject, batch, study, or time-ordered resampling.
Cross-validated fitting	`fit_resample()`	Fold predictions, fold metrics, aggregated performance summaries, preprocessing state, and refit metadata.
Nested tuning	`tune_resample()`	Nested cross-validation for hyperparameter tuning with leakage-aware outer and inner folds.
Leakage auditing	`audit_leakage()`	Permutation-gap summaries, batch or study association tests, target scans, duplicate detection, and mechanism summaries.
Audit reporting	`audit_report()`	Self-contained HTML summaries suitable for documentation and sharing.
Inflation quantification	`delta_lsi()`	Repeat-level comparison of naive and guarded pipelines, including robust inflation estimates and randomization-based inference.
Interoperability	`guard_to_recipe()`, `as_rsample()`	Bridges to **recipes** and **rsample** workflows.

bust point estimates, confidence intervals, and sign-flip randomization tests.

Table **??** lists the primary user-facing functions and their roles in the workflow.

4. Leakage-Aware Modeling Workflow

This section illustrates a typical workflow using a small synthetic binary classification example. The same workflow structure can be adapted to multiclass, regression, and survival tasks, although the available metrics and some audit components differ across settings. The underlying evaluation logic follows standard resampling principles, but adapts them to structured biomedical data in which the relevant unit of independence may be subject, batch, study, or time rather than the individual row (**???**).

4.1. Constructing a split plan

The first step is to define the dependence structure that the resampling design should respect. In the example below, repeated measurements are nested within subject, so a subject-grouped split plan is used. The code mirrors the current package API and can be adapted to `batch_blocked`, `study_loocv`, or `time_series` settings by changing the resampling mode and the corresponding grouping or time-index columns.

```
library(bioLeak)
library(parsnip)

set.seed(1)
n_subject <- 40
rep_per_subject <- 3
n <- n_subject * rep_per_subject

subject <- rep(seq_len(n_subject), each = rep_per_subject)
batch <- rep(seq_len(6), length.out = n)
x1 <- rnorm(n)
x2 <- rnorm(n)
x3 <- rnorm(n)
subject_signal <- rnorm(n_subject)
outcome <- factor(
  ifelse(runif(n) < plogis(subject_signal[subject]), "case", "control"),
  levels = c("control", "case")
)

df <- data.frame(subject, batch, outcome, x1, x2, x3)

splits <- make_split_plan(
  df,
  outcome = "outcome",
  mode = "subject_grouped",
  group = "subject",
  v = 5,
  stratify = TRUE,
  seed = 1
)

splits

LeakSplits object (mode = subject_grouped, v = 5, repeats = 1)
Outcome: outcome | Stratified: TRUE | Nested: FALSE
-------------------------------------------------------
  repeat fold n_train n_test
       1    1      96     24
       1    2      96     24
       1    3      96     24
       1    4      96     24
       1    5      96     24
-------------------------------------------------------
Total folds: 5 | Hash: a1b2c3d4e5f6
```

The split plan is created successfully and yields five subject-grouped folds of equal size. Because all repeated measurements from a given subject are assigned to the same fold, the

resampling design prevents subject overlap between training and test sets. In this synthetic example, the object serves as the common evaluation structure for both the guarded and intentionally leaky fits considered below.

The resulting split object stores fold assignments together with the resampling settings used to generate them, including the split mode, outcome, and stratification status. For large datasets, `compact = TRUE` stores fold assignments as a per-row fold vector rather than retaining explicit index lists for each fold, reducing memory use by roughly v-fold (e.g., from five copies of row indices to one integer vector).

4.2. Defining guarded preprocessing

`fit_resample()` accepts a native guarded preprocessing specification or a **recipes** recipe object. Under the native interface, each preprocessing step is estimated on the training fold only and the resulting parameters are then applied to the corresponding test fold. Supported operations include imputation, normalization, filtering, and feature selection. In the example below, median imputation and z-score normalization are performed separately within each training fold.

```
spec <- logistic_reg(mode = "classification") |>
  set_engine("glm")

fit_guarded <- fit_resample(
  df,
  outcome = "outcome",
  splits = splits,
  learner = spec,
  metrics = "auc",
  preprocess = list(
    impute = list(method = "median"),
    normalize = list(method = "zscore")
  ),
  refit = FALSE,
  seed = 1
)

summary(fit_guarded)
```

```
============================
 bioLeak Model Fit Summary
============================

Task: binomial
Outcome: outcome
Positive class: case
Learners: logistic_reg/glm
Total folds: 5
Fold status: 5 success, 0 skipped, 0 failed
```

```
Refit performed: No
Hash: 7a3f2e1b4c9d

Cross-validated metrics (mean +/- SD):
           learner auc_mean auc_sd auc_ci_lo auc_ci_hi
1 logistic_reg/glm    0.542  0.102      0.34      0.74
```

The guarded fit completes successfully across all five folds and yields only modest discrimination (mean AUC 0.54). In this synthetic example, that result is expected because the predictors are mostly noise and no label-derived feature has been introduced. This fitted object therefore serves as the guarded baseline for the leaky comparator examined in the following subsection.

The same interface extends to multiclass classification, regression, and survival settings, although the available metrics and supported options differ by task type. For survival analysis, the outcome is specified using time and event columns, and concordance-based evaluation is supported. For continuous regression, common metrics include `rmse`, `mae`, and R^2. Survival support in the current version covers `fit_resample()` and `delta_lsi()` with concordance metrics; however, nested tuning, target scanning, and permutation-refit audits are not yet available for survival tasks. Because the simulation study and case study both use binary classification, survival workflows are not demonstrated in this paper.

4.3. Training naive and guarded comparators

The package is most informative when a guarded analysis is compared with a plausible naive workflow or with a deliberately constructed leakage scenario. Such comparisons are useful for sensitivity analysis and provide the basis for the `delta_lsi()` procedure described in Section **??**. In the example below, a label-derived subject-level feature is added to create an intentionally leaky comparator while keeping the same grouped split structure. The resulting fitted object is used in the following subsection to quantify performance inflation relative to the guarded analysis.

```
df_leaky <- within(df, {
  leak_subject <- ave(as.numeric(outcome == "case"), subject, FUN = mean)
})

fit_leaky <- fit_resample(
  df_leaky,
  outcome = "outcome",
  splits = splits,
  learner = spec,
  metrics = "auc",
  preprocess = list(
    impute = list(method = "none"),
    normalize = list(method = "none")
  ),
  refit = FALSE,
  seed = 1
```

```
)

summary(fit_leaky)

===========================
 bioLeak Model Fit Summary
===========================

Task: binomial
Outcome: outcome
Positive class: case
Learners: logistic_reg/glm
Total folds: 5
Fold status: 5 success, 0 skipped, 0 failed
Refit performed: No
Hash: h9bcac8d4

Cross-validated metrics (mean +/- SD):
           learner auc_mean auc_sd auc_ci_lo auc_ci_hi
1 logistic_reg/glm     0.92  0.054      0.82      1.00

Audit overview:
 fold n_train n_test          learner features_final
    1      96     24 logistic_reg/glm              5
    2      96     24 logistic_reg/glm              5
    3      96     24 logistic_reg/glm              5
    4      96     24 logistic_reg/glm              5
    5      96     24 logistic_reg/glm              5
```

The intentionally leaky comparator yields a markedly higher cross-validated AUC than the guarded fit shown earlier. Because the grouped split structure is unchanged, this increase is attributable to the label-derived subject-level feature rather than to a change in resampling design. The example therefore illustrates how leakage can inflate apparent predictive performance even when fold construction itself is appropriate.

4.4. Auditing a fitted workflow

After cross-validated predictions have been generated, `audit_leakage()` can be used to summarize multiple forms of audit evidence from the fitted workflow and, optionally, from an aligned reference matrix `X_ref`. The reference matrix enables target-leakage scans and near-duplicate detection. In grouped designs, the same interface can also be used to assess whether performance remains above chance under label permutation and whether fold assignments are associated with batch or study structure.

```
audit_guarded <- audit_leakage(
  fit_guarded,
  metric = "auc",
```

```
  B = 50,
  perm_stratify = TRUE,
  X_ref = df[, c("x1", "x2", "x3")],
  batch_cols = "batch",
  return_perm = TRUE
)

summary(audit_guarded)
```

```
==============================
 bioLeak Leakage Audit Summary
==============================

Task: binomial | Outcome: outcome | Splitting mode: subject_grouped
Hash: 7a3f2e1b4c9d | Folds: 5 | Repeats: 1

Label-Permutation Association Test:
  Method: fixed predictions
  Observed metric: 0.5417
  Permuted mean +/- SD: 0.5012 +/- 0.0389
  Gap: 0.0405 (larger gap = stronger non-random signal)

Batch / Study Association:
  batch (repeat 1): Chi^2 = 4.123 (df = 4), p = 0.3897

Target Leakage Scan:
  Features checked: 3 | Flagged (score >= 0.90): 0

Multivariate Target Scan: not available.

Near-Duplicate Samples:
  No near-duplicates detected.

Mechanism Risk Assessment:
  mechanism_class      flagged evidence
  subject_overlap      FALSE   OK: subject-grouped splits
  batch_confounded     FALSE   OK: batch Chi^2 not significant
  preprocessing_leak   FALSE   OK: guarded preprocessing
  target_leakage       FALSE   OK: no features flagged

Interpretation:
  Modest non-random signal. No leakage indicators flagged.
```

In this example, the audit detects modest predictive signal but does not flag subject overlap, batch-related structure, target leakage, or near-duplicate samples. The guarded fit therefore appears consistent with non-random but not obviously leakage-driven discrimination.

The resulting `LeakAudit` object stores permutation summaries, feature-level target-association results, duplicate-detection output, and the audit settings used to generate them. The permutation module is diagnostic rather than dispositive: a positive gap suggests predictive signal beyond chance, but the remaining audit components are needed to judge whether that signal may be inflated by leakage.

Practical guidance on permutation calibration. By default, `fit_resample()` stores the original data and learner specification inside the `LeakFit` object (`store_refit_data = TRUE`). When `audit_leakage()` is subsequently called with the default `perm_refit = "auto"`, it detects this stored specification and activates refit-based permutations, in which the model is retrained under each label permutation. This yields a well-calibrated null distribution with type I error at the nominal level. The fixed-prediction alternative (`perm_refit = FALSE`), which the simulations in Section **??** use for computational reasons, is faster but slightly liberal (empirical type I error of 6–10% depending on feature dimension). Users who set `perm_refit = FALSE` or `store_refit_data = FALSE` should be aware of this trade-off; the package emits an informational message when auto mode falls back to fixed predictions.

4.5. Generating an HTML audit report

The audit can be rendered as a standalone HTML report for documentation or sharing. The report summarizes all audit components described above.

```
report_file <- audit_report(
  audit_guarded,
  output_file = "bioLeak_audit_report.html",
  output_dir = tempdir(),
  quiet = TRUE
)
```

The returned value is the path to the generated HTML file, which can then be opened in a browser or archived alongside other analysis outputs.

4.6. Putting the workflow together

The previous subsections present each stage separately. In practice, the full workflow is usually executed as a single script. The following code block reuses the simulated data frame `df` from Section **??** and chains every stage into one end-to-end sequence.

```
## Assumes df and spec from Sections 4.1-4.2 are in scope
dat <- df

plan <- make_split_plan(
  dat,
  outcome = "outcome",
  mode = "subject_grouped",
  group = "subject",
  v = 5,
```

```
  stratify = TRUE,
  seed = 1
)

spec <- logistic_reg(mode = "classification") |>
  set_engine("glm")

fit_guarded <- fit_resample(
  dat,
  outcome = "outcome",
  splits = plan,
  learner = spec,
  metrics = "auc",
  refit = FALSE,
  seed = 1
)

dat_naive <- within(dat, {
  leak_subject <- ave(as.numeric(outcome == "case"), subject, FUN = mean)
})

fit_leaky <- fit_resample(
  dat_naive,
  outcome = "outcome",
  splits = plan,
  learner = spec,
  metrics = "auc",
  refit = FALSE,
  seed = 1
)

audit <- audit_leakage(
  fit_guarded,
  metric = "auc",
  B = 50,
  perm_stratify = TRUE,
  batch_cols = "batch",
  X_ref = dat[, c("x1", "x2", "x3")]
)

delta <- delta_lsi(
  fit_leaky = fit_leaky,
  fit_guarded = fit_guarded,
  metric = "auc",
  seed = 1
)
```

```
report_file <- audit_report(
  audit,
  output_file = "bioLeak_audit_report.html",
  output_dir = tempdir(),
  quiet = TRUE
)
```

The split plan defines the dependence structure, `fit_resample()` trains the guarded and intentionally leaky pipelines under that design, `audit` stores diagnostic evidence for the guarded fit, `delta` summarizes performance inflation between the two analyses, and `audit_report()` produces a shareable HTML summary. This end-to-end pattern extends to multiclass, regression, and survival settings with task-appropriate learners and metrics.

4.7. Leakage-safe hyperparameter tuning

`tune_resample()` performs nested cross-validation so that hyperparameter selection remains inside the outer resampling loop. The example below uses a tunable **glmnet** specification together with a nested split plan. This design avoids tuning on the same assessment data used for outer-loop evaluation.

```
library(recipes)
library(tune)

nested_splits <- make_split_plan(
  df,
  outcome = "outcome",
  mode = "subject_grouped",
  group = "subject",
  v = 3,
  nested = TRUE,
  seed = 14
)

rec <- recipe(outcome ~ x1 + x2 + x3, data = df) |>
  step_impute_median(all_numeric_predictors()) |>
  step_normalize(all_numeric_predictors())

spec_tune <- logistic_reg(
  penalty = tune(),
  mixture = 1,
  mode = "classification"
) |>
  set_engine("glmnet")

tuned <- tune_resample(
  df,
  outcome = "outcome",
```

```
  splits = nested_splits,
  learner = spec_tune,
  preprocess = rec,
  grid = 5,
  metrics = c("auc", "accuracy"),
  selection = "one_std_err",
  refit = TRUE,
  seed = 14
)
```

The resulting object stores outer-fold performance summaries together with the tuning results needed to identify the selected hyperparameter values under nested resampling.

When a `recipe` or `workflow` is supplied, **bioLeak** applies guard checks to reduce the risk of preprocessing steps being estimated outside the intended fold structure. The package also provides `guard_to_recipe()`, which translates a guarded preprocessing specification into an approximate **recipes** pipeline so that users who prefer the **tidymodels** ecosystem can apply the same transformations; the function maps imputation, normalization, and filtering steps to their **recipes** equivalents. At present, `tune_resample()` supports classification and regression workflows, but not survival outcomes.

4.8. Practical workflow variants

The same workflow generalizes to the other supported split modes. With `batch_blocked`, all records from the same batch are assigned to the same fold. With `study_loocv`, one study is held out at a time, which is often appropriate for consortium datasets or data aggregated from public resources. With `time_series`, observations are ordered chronologically and training is restricted to records that precede the test block, optionally with additional separation between training and evaluation periods. A `combined` mode applies multiple constraints simultaneously (e.g., subject grouping with batch blocking). In all cases, the core principle remains the same: resampling should represent the intended deployment setting rather than a convenient random partition of rows.

5. Quantifying Leakage Inflation (`Delta LSI`)

bioLeak provides `delta_lsi()` to compare a naive pipeline with a guarded one when both are fitted on the same repeated resampling design. The goal is not to decide whether one pipeline is "better" in a generic sense, but to quantify how much apparent performance changes when leakage controls are introduced. This focus is consistent with the broader cross-validation literature, which emphasizes that resampling estimates are themselves random quantities and that their uncertainty is nontrivial to characterize (**??**).

Suppose repeated cross-validation yields R repeat-level summaries. Let $M_{\text{naive},r}$ and $M_{\text{guarded},r}$ denote the repeat-level performance estimates for repeat r. For higher-is-better metrics such as AUC or accuracy, the repeat-level inflation score is

$$\Delta_r = M_{\text{naive},r} - M_{\text{guarded},r}, \qquad r = 1, \ldots, R.$$

Positive values therefore indicate that the naive pipeline appears more optimistic than the guarded one and, under a properly paired comparison, are consistent with leakage-related

performance inflation. For lower-is-better metrics such as RMSE or log-loss, `delta_lsi()` applies a sign flip internally through the `higher_is_better` argument so that positive values still denote inflation.

In the implementation, fold-level metric values are aggregated to a repeat level using test-set-size weights. Let μ_r denote the resulting repeat summary for one pipeline. The vector $\boldsymbol{\Delta} = (\Delta_1, \ldots, \Delta_R)^\top$ is then summarized in two ways. The first is the arithmetic mean,

$$\widehat{\Delta}_{\text{metric}} = \frac{1}{R} \sum_{r=1}^{R} \Delta_r,$$

which is easy to interpret directly in the units of the metric. The second is the robust summary reported as `delta_lsi`. Intuitively, this estimator behaves like the arithmetic mean for typical repeats but automatically downweights any repeat whose inflation score is unusually large or small, providing robustness when one or two folds are unstable for idiosyncratic reasons. Formally,

$$\widehat{\Delta}_{\text{LSI}} = \arg\min_{\mu} \sum_{r=1}^{R} \rho_{1.345} \left(\frac{\Delta_r - \mu}{\widehat{\sigma}} \right),$$

where $\rho_{1.345}$ is Huber's loss function and $\widehat{\sigma} = 1.4826 \times \text{MAD}(\boldsymbol{\Delta})$ is a fixed robust scale estimate. The minimization is solved by iteratively reweighted least squares (IRLS) (**??**).

Confidence intervals are computed with a bias-corrected and accelerated (BCa) bootstrap when the effective number of paired repeats is at least ten. The package reports a 95% interval for both `delta_lsi` and the arithmetic mean. Hypothesis testing is based on a sign-flip randomization test. Under the null hypothesis of no systematic inflation, the sign of each Δ_r is exchangeable. The test statistic is $T = R^{-1} \sum_{r=1}^{R} s_r \Delta_r$, where $s_r \in \{-1, +1\}$ are sign-flip indicators and the observed value uses $s_r = 1$ for all r. For $R \leq 15$, the package enumerates all 2^R sign combinations exactly. For larger R, Monte Carlo sign-flips are sampled and the Phipson–Smyth correction is applied so that the reported p-value is never zero (**?**). When `exchangeability = "blocked_time"`, a block sign-flip procedure preserves serial autocorrelation under the null. Two further options (`"by_group"`, `"within_batch"`) are accepted but currently fall back to the iid procedure with a warning.

The design is only fully inferential when the two compared fits are paired. In practice this means they must share the same fold structure, not merely the same number of repeats. If fold membership differs, the package falls back to an unpaired comparison of repeat-level summaries and suppresses formal inference. This distinction is important. If the user changes both the predictors and the resampling design, the resulting performance difference cannot be attributed solely to leakage control.

The returned `LeakDeltaLSI` object stores point estimates, intervals, the sign-flip p-value, and repeat-level summaries for both pipelines. The package groups inferential output into four tiers:

1. `D_insufficient`: fewer than five effective paired repeats or an unpaired comparison; point estimates only.

2. `C_signflip`: at least five paired repeats; point estimates and a sign-flip p-value.

3. `B_signflip_ci`: at least ten paired repeats; point estimates, p-value, and BCa intervals.

4. `A_full_inference`: at least twenty paired repeats with finite intervals and p-value.

A typical call compares a naive and a guarded fit built from the same `LeakSplits` object:

```
inflation <- delta_lsi(
  fit_leaky = fit_leaky,
  fit_guarded = fit_guarded,
  metric = "auc",
  M_boot = 2000,
  M_flip = 10000,
  seed = 42
)

summary(inflation)
```

The output below is from the case-study comparison described in Section **??**, which uses 20 paired batch-blocked repeats:

```
=====================================
 bioLeak Delta LSI Summary
=====================================

Metric:            auc
Exchangeability:   iid
Inference tier:    A_full_inference
R_eff:             20  (R_leaky=20, R_guarded=20, paired=TRUE)

Leaky pipeline:    0.791
Guarded pipeline:  0.611

Point estimates (leaky - guarded; positive = leakage inflation):
  delta_metric:  0.180  (raw metric difference)
  95% BCa CI:    [0.172, 0.186]
  delta_lsi:     0.181  (Huber-robust)
  95% BCa CI:    [0.174, 0.187]

Hypothesis test  [H0: no systematic inflation; paired repeat signs exchangeable]:
  Sign-flip p:   0.0001 ***

Inference valid:   YES (tier A)
```

In practice, `delta_lsi()` is most useful when an analyst wants to measure the sensitivity of a result to specific leakage controls, such as subject grouping, batch blocking, or removal of explicit leak-prone features. Section **??** evaluates the audit framework across a factorial simulation design, and Section **??** demonstrates a paired `delta_lsi()` comparison on real data. A small power simulation (`run_delta_lsi.R`; $n = 200$, $p = 20$, 50 seeds, 20-repeat 5-fold CV with a `peek_norm` leakage feature at $\sigma = 0.3$) yielded 100% rejection at $\alpha = 0.05$

with a mean $\widehat{\Delta}_{\text{metric}} = 0.197$, confirming that the sign-flip test reliably detects leakage-induced inflation in this setting. A complementary null calibration confirmed nominal Type I error control. In each of 50 replications, $R = 20$ independent datasets were drawn from the same population and both arms received their own exchangeable noise feature (pure $\mathcal{N}(0, 1)$, uncorrelated with the outcome). The observed rejection rate was 8.0% at $\alpha = 0.05$ ($p = 0.24$ by a binomial test against the nominal 5%, non-significant), with a mean $\widehat{\Delta}_{\text{metric}}$ of 0.0001 and balanced sign proportions across repeats (replication script: `run_delta_lsi.R`, Part C). Under weaker leakage or smaller samples, power would be lower; a comprehensive power characterization is left for future work.

6. Simulation Study

6.1. Design

An important interpretive caveat applies throughout this section. All permutation-gap analyses use `perm_refit = FALSE`, meaning that the model is fitted once on the original labels and the permutation procedure shuffles labels against fixed predictions. This design tests whether the prediction–label association exceeds chance levels, but it does not distinguish genuine predictive signal from signal attributable to leakage. As a result, the clean baseline (`none`) can also reject at high rates when real signal is present ($s > 0$). Leakage-specific detection is therefore most informative at $s = 0$, where the outcome contains no genuine predictive signal and any above-chance performance under the leakage conditions must arise from the injected leakage mechanism.

The manuscript simulations were generated by a dedicated script (`run_simulation.R`) whose settings differ in several respects from the package helper `simulate_leakage_suite()`. These choices were made to create leakage signals of graded strength for the paper-specific experiments. In particular, the manuscript script generates peek-norm leakage as $y + \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, 0.09)$, implements look-ahead leakage by shifting a continuous biomarker rather than the binary outcome, and adds AR(1) noise ($\rho = 0.9$) to the latent linear predictor when $s > 0$. Readers who wish to reproduce the manuscript results exactly should use the paper script rather than the package helper.

The main simulation follows a full factorial design with five leakage settings (none, subject overlap, batch confounded, peek normalization, and look-ahead), four sample sizes ($n \in \{100, 250, 500, 1000\}$), three feature dimensions ($p \in \{10, 50, 100\}$), three signal levels ($s \in \{0, 0.5, 2.0\}$), and fifty seeds per configuration, yielding 9,000 simulated datasets in total.

For each dataset, predictors were generated as Gaussian variables and the binary outcome was sampled from a probit link applied to a latent linear predictor. When $s = 0$, the outcome contained no genuine predictive signal. Under the clean condition (`none`), this setting provides a null baseline; under the leakage conditions, any above-chance performance is attributable to the injected leakage mechanism rather than to real signal. When $s > 0$, signal entered through the first few predictors, and autoregressive noise was added to the latent process.

Leakage was injected as follows:

- `subject_overlap`: a subject-level mean of the binary outcome was added as a predictor;

- `batch_confounded`: batch membership was made outcome dependent and the batch-wise mean outcome was added as a predictor;
- `peek_norm`: a noisy continuous version of the outcome was added as a globally computed feature; and
- `lookahead`: a future biomarker value was shifted into the current row. Because the shifted covariate carries outcome information only through the latent signal, this mechanism is structurally null at $s = 0$; it is included primarily to evaluate detection under the $s > 0$ conditions.

Preprocessing leakage (Section **??**, Section 2.1) is not directly simulated as a separate mechanism. The guarded pipeline applies train-fold-only preprocessing by construction, so the effect of global-vs-fold-wise preprocessing is absorbed into the package's default behavior rather than isolated as a controlled factor. A dedicated simulation comparing fold-wise and global normalization on correlated-feature designs would be a valuable extension but is beyond the scope of the present study.

All main-simulation analyses used the same five-fold subject-grouped cross-validation design in order to hold the evaluation structure fixed while varying the injected leakage mechanism. The learner was **glmnet**, with median imputation, z-score normalization, no feature selection, and a permutation-gap audit using $B = 200$ and `perm_refit = FALSE`. The reported summaries include the observed AUC, the permutation-null mean, the permutation gap, and the permutation p-value.

Two supplementary analyses refine this design. One evaluates the same general leakage mechanisms under four split modes (`subject_grouped`, `batch_blocked`, `study_loocv`, and `time_series`) at a fixed setting ($n = 500, p = 20, s = 1.0$). A second supplementary analysis evaluates the target-scanning components at the same fixed setting.

6.2. Main simulation summaries

The main simulation results, generated by `run_simulation.R`, are summarized in Tables **??** and **??**. The two tables separate signal-bearing settings ($s > 0$) from the null setting ($s = 0$) where leakage-specific detection can be interpreted more directly.

Figure **??**(c) shows rejection rates at $s = 0$ (no real signal) for all mechanisms across feature dimensions. This isolates leakage detection from genuine predictive signal. Peek normalization is detected at 100% regardless of p, while subject overlap and batch confounding are detected at 92–99%. In contrast, look-ahead and the clean baseline both remain near the nominal level (6–10%), confirming that look-ahead carries no outcome information at $s = 0$ by design.

Null calibration of the fixed-prediction permutation test. For the clean baseline, the overall false rejection rate with `perm_refit = FALSE` was **7.8%** at $\alpha = 0.05$ (8.0% at $n = 100$, 9.3% at $n = 250$, 8.0% at $n = 500$, 6.0% at $n = 1000$). The inflation scales with feature dimension: **6.0%** at $p = 10$ rising to **10.5%** at $p = 100$, and diminishes with sample size. This mild liberality is a known property of fixed-prediction permutation tests: the model captures weak noise correlations during training, producing a small positive gap even when no true signal exists. In normal usage this inflation does not arise, because the package

Table 2: Rejection rates (permutation-gap $p < 0.05$) at $s > 0$, pooled over p and s.

Mechanism	$n = 100$	$n = 250$	$n = 500$	$n = 1000$
None (clean baseline)	0.57 (.03)	0.82 (.02)	0.98 (.01)	1.00
Subject overlap	0.93 (.01)	1.00	1.00	1.00
Batch confounded	0.95 (.01)	1.00	1.00	1.00
Peek normalization	1.00	1.00	1.00	1.00
Look-ahead	0.65 (.03)	0.95 (.01)	1.00	1.00

default `perm_refit = "auto"` activates refit-based permutations whenever `fit_resample()` has stored the original data and learner (the default `store_refit_data = TRUE`); see the practical guidance in Section **??**. The simulations in this paper use explicit `perm_refit = FALSE` because the goal is to characterize the permutation-gap statistic as a fast association test under controlled leakage, not to perform full refit-based null calibration. A refit-based evaluation with $B = 500$ would require 500 full model fits per task, increasing the total runtime from roughly three hours to several hundred hours—prohibitive for a 9,000-task factorial design.

For leakage-injected settings with $s > 0$, the permutation-gap statistic was usually significant. However, with `perm_refit = FALSE` the permutation-gap test detects prediction-label association rather than leakage specifically. Figure **??**(b) shows how observed AUC varies with signal strength for each mechanism. At $s = 0$ (no real signal), the clean baseline sits at chance (≈ 0.50), while `peek_norm` already reaches 0.99 — demonstrating that this leakage mechanism creates apparent performance from nothing. Subject overlap and batch confounding similarly inflate AUC above chance at $s = 0$ (≈ 0.72–0.73), whereas look-ahead is indistinguishable from clean. As s increases, all mechanisms converge because genuine signal dominates.

Turning to rejection rates, the clean baseline (`none`) also rejects at high rates when real signal is present: 84.2% overall at $s > 0$, reaching 98% at $n = 500$ and 100% at $n = 1000$. Table **??** includes this baseline alongside the leakage-injected runs so that the increment attributable to leakage is visible. Pooled over feature dimension and the two positive signal levels, the weakest leaky setting is `lookahead` at $n = 100$ (0.65), which is close to the clean baseline at the same n (0.57). All other mechanisms exceed the clean baseline by a larger margin at $n = 100$ and converge to 1.00 by $n = 500$. Each cell in Table **??** pools 300 runs (3 feature dimensions $\times$ 2 signal levels $\times$ 50 seeds); standard errors are binomial and shown in parentheses. Cells at 1.00 have zero Wald standard error; the Wilson 95% lower bound is 0.99 for 300/300.

These values should be interpreted carefully. With `perm_refit = FALSE`, the permutation-gap test detects non-random predictive signal rather than leakage in isolation, so the high baseline rejection rate at $s > 0$ means that absolute rejection rates cannot be read as leakage-detection power. A more informative view comes from the $s = 0$ condition, where no real signal exists and the only source of above-chance performance is the injected leakage. Table **??** shows the leakage-specific detection rates under this design. Three of the four mechanisms are detected reliably by $n = 250$; look-ahead is not detectable at $s = 0$ because the shifted biomarker carries no outcome information when the latent signal is absent. Each cell pools 150 runs (3 feature dimensions $\times$ 50 seeds); standard errors and Wilson bounds follow the same conventions as Table **??** (Wilson 95% lower bound 0.98 for 150/150).

Table 3: Leakage-specific detection at $s = 0$ (no real signal), pooled over p.

Mechanism	$n = 100$	$n = 250$	$n = 500$	$n = 1000$
None (clean baseline)	0.08 (.02)	0.09 (.02)	0.08 (.02)	0.06 (.02)
Subject overlap	0.85 (.03)	1.00	1.00	1.00
Batch confounded	0.89 (.03)	1.00	1.00	1.00
Peek normalization	1.00	1.00	1.00	1.00
Look-ahead	0.09 (.02)	0.09 (.02)	0.09 (.02)	0.06 (.02)

The clean-baseline rejection rates of 6–10% at $s = 0$ exceed the nominal 5% level. This is consistent with the known liberal calibration of the fixed-prediction permutation test (`perm_refit = FALSE`); under refit-based permutation, these rates would be closer to the nominal level (Section **??**).

This two-table presentation separates the permutation test's role as a general performance check ($s > 0$) from its role as a leakage detector ($s = 0$). The package combines the permutation module with targeted diagnostics for confounding, duplicates, and target leakage precisely because no single component is leakage-specific in all settings.

Figure **??**(a) shows the observed AUC for each mechanism across sample sizes (averaged over $s > 0$ conditions). Peek normalization inflates AUC to near 1.0 regardless of n, while subject overlap and batch confounding produce moderate inflation that narrows as n grows and genuine signal dominates. Look-ahead leakage sits only slightly above the clean baseline, consistent with the weak temporal correlation in this simulation design. To quantify inflation directly, panel (d) compares each leakage mechanism with the clean baseline at matched (n, s) values. Averaged over the stored configurations with $s > 0$, the mean observed AUC in clean settings was 0.75. Relative to that clean baseline, the mean inflation in observed AUC was 0.089 for `subject_overlap`, 0.093 for `batch_confounded`, 0.244 for `peek_norm`, and 0.028 for `lookahead`. In these stored results, explicit global peek-type leakage produces the largest inflation, whereas the look-ahead construction yields the smallest average AUC increase. This difference is attributable to the particular manuscript design in `run_simulation.R`; it should not be read as a universal ranking of leakage severity.

6.3. Supplementary split-mode and target-scan analyses

Table **??** summarizes the supplementary split-mode analysis from `run_supplementary.R` in the replication materials. As with the main simulation, the split-mode experiment was run at both $s = 0$ (leakage-specific) and $s = 1.0$ (signal present). At $s = 1.0$, the clean baseline rejects in all 50/50 runs for every split mode, so the $s = 1.0$ results cannot isolate leakage. Table **??** therefore reports the $s = 0$ condition, where the only source of above-chance performance is the injected leakage. Under all four split modes, `subject_overlap`, `batch_confounded`, and `peek_norm` are detected with high power, with the notable exception of `batch_confounded` under `batch_blocked` mode (0.02), where the blocking variable aligns with the confounding structure. Look-ahead leakage is largely undetectable at $s = 0$ (rejection rates 0.02–0.04 across modes), which is expected: this mechanism leaks future values of a continuous biomarker that carries signal only when real signal is present. Each cell is based on 50 seeds; standard errors in parentheses. Cells at 1.00 have Wilson 95% lower bound 0.93 for 50/50.

The reduced detection of `batch_confounded` leakage under `batch_blocked` mode is an ex-

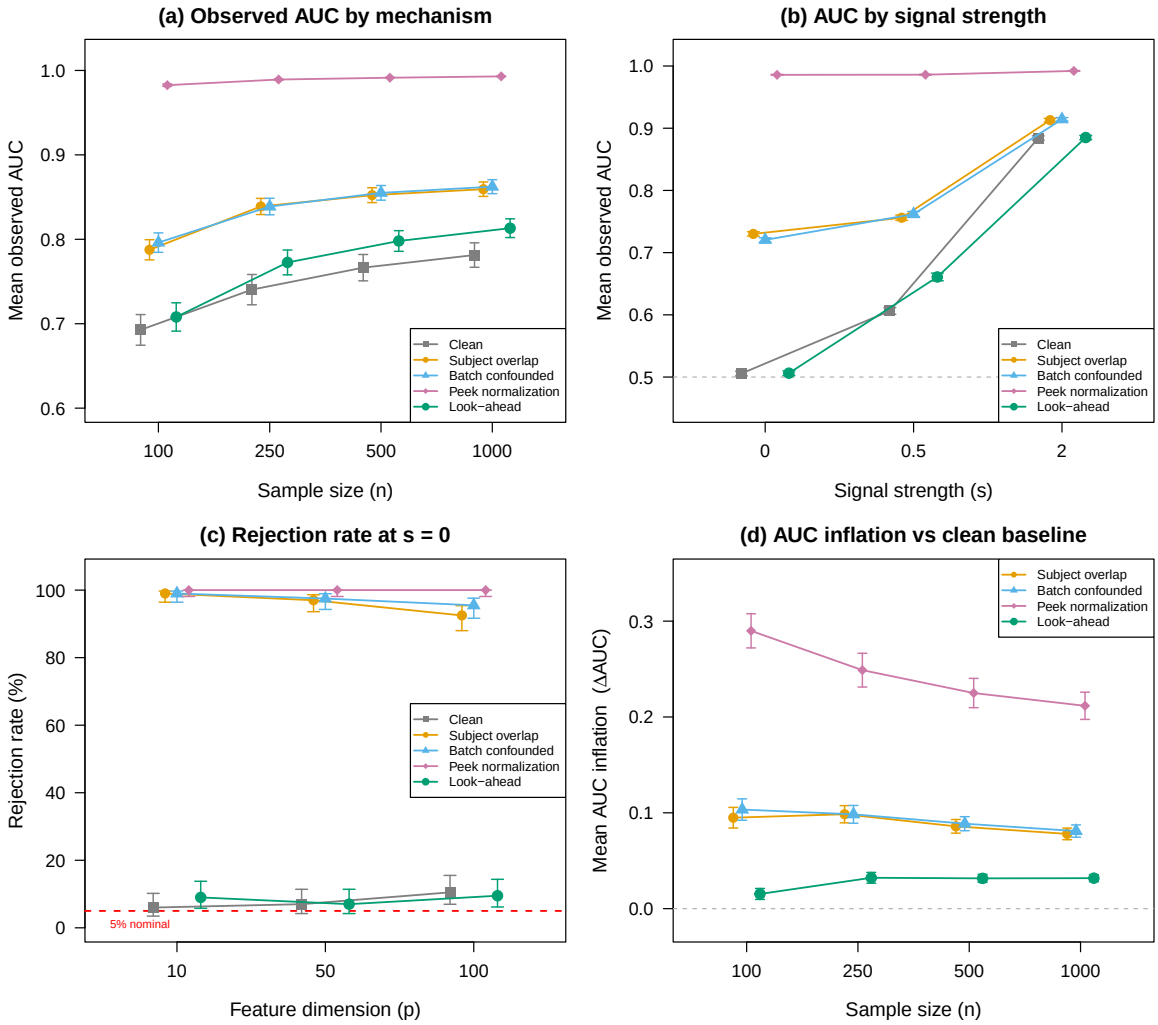

Figure 2: Simulation results across leakage mechanisms. (a) Observed AUC across sample sizes ($s > 0$). (b) AUC across signal strengths (all s). (c) Rejection rate across feature dimensions ($s = 0$). (d) AUC inflation relative to clean baseline ($s > 0$). Error bars are 95% confidence intervals: Student-t for the AUC panels (a, b, d); Wilson for the binomial rejection rate in panel (c). Intervals may be too narrow to be visible at highly precise points. Points are horizontally jittered to avoid overlap.

Table 4: Supplementary split-mode analysis: rejection rates (permutation-gap $p < 0.05$) at ($n = 500, p = 20, s = 0$).

Mode	**None**	**Subj. overlap**	**Batch conf.**	**Peek norm.**	**Look-ahead**
Subj. grouped	0.06 (.03)	1.00	0.98 (.02)	1.00	0.04 (.03)
Batch blocked	0.04 (.03)	1.00	0.02 (.02)	1.00	0.02 (.02)
Study LOOCV	0.06 (.03)	1.00	0.98 (.02)	1.00	0.04 (.03)
Time series	0.02 (.02)	1.00	0.98 (.02)	1.00	0.02 (.02)

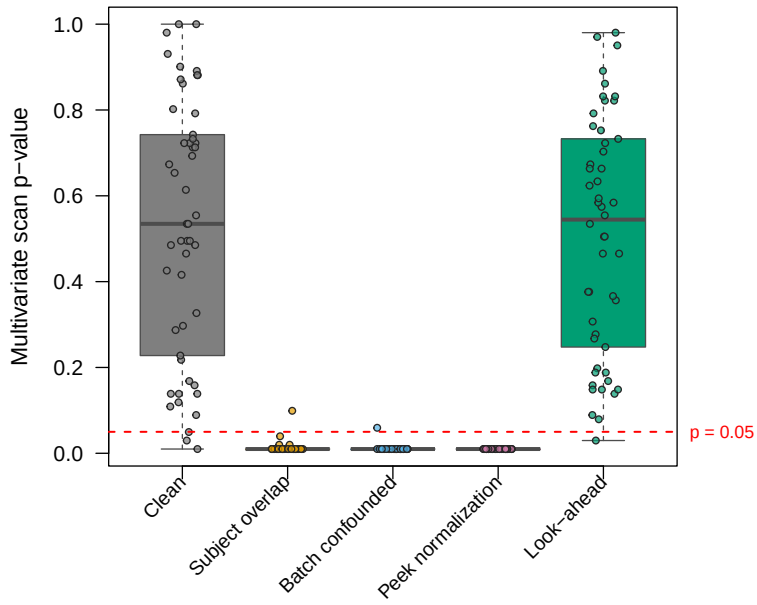

Figure 3: Multivariate target-scan p-values at $s = 0$.

pected interaction: when the test fold contains samples from a single batch, confounding between batch and outcome is inherently difficult to detect via permutation because the batch structure is held constant. This illustrates a key design lesson: the permutation-gap test's sensitivity depends on the interplay between the leakage mechanism and the resampling geometry. The exercise reinforces the main software point of **bioLeak**: the choice of split design is part of the model specification, and combining permutation tests with targeted diagnostics (such as the target scan) provides more robust detection.

The supplementary target-scan analysis, also from `run_supplementary.R`, provides a more granular picture. At $s = 0$ (no real signal), the univariate target scan flagged the injected leakage feature in all 50/50 `peek_norm` runs and in none of the `subject_overlap`, `batch_confounded`, `lookahead`, or clean runs at the threshold used in the stored script (`target_threshold = 0.9`). Averaged over all leaky conditions, this corresponds to a 25% success rate because only one of the four injected mechanisms satisfied the conservative univariate threshold. The multivariate scan at $s = 0$ rejected in 74% of leaky runs, with a 6% false-positive rate in the clean baseline (Figure **??**)—consistent with the nominal $\alpha = 0.05$ level. This contrasts with the $s = 1.0$ evaluation, where the multivariate scan rejects in all conditions (including the clean control) because the real signal features dominate the PCA–GLM statistic. The $s = 0$ results confirm that the multivariate scan has genuine leakage-detection power when confounding with real signal is removed. Nonetheless, the univariate and multivariate scans should be interpreted jointly, and with knowledge of the reference matrix supplied to `X_ref`.

7. Case Study

7.1. Data and workflow

The case study is based on `run_casestudy.R` in the replication materials. The script draws data from the **curatedOvarianData** Bioconductor package and constructs a binary three-year overall survival endpoint from studies containing both `days_to_death` and `vital_status`. After intersecting feature sets across eligible studies and retaining the 2,000 most variable common genes, the final analysis dataset contains $N = 2{,}015$ samples from $S = 14$ studies and $G = 2{,}000$ genes. Study sizes range from 50 to 376 samples, with a median of 125. Multi-study gene-expression prediction has long served as a test case for both high-dimensional classifiers and evaluation pitfalls in biomedical machine learning (**???**).

Four workflows were evaluated to permit a clean decomposition of the performance gap. The *guarded* workflow used `study_loocv`, median imputation, z-score normalization, t-test feature selection with the top 100 genes per fold, and a **glmnet** learner. The *leaky* comparator used standard five-fold row-wise evaluation implemented via a `subject_grouped` split on a dummy row identifier, the same learner, and three deliberately leak-prone features: a per-study outcome mean, the binary outcome with additive Gaussian noise ($\sigma = 1.0$), and a globally standardized first principal component.

Because the guarded and leaky pipelines differ in three ways simultaneously (feature selection, split design, and leakage features), attributing the AUC gap to any single factor requires intermediate arms. Two sensitivity pipelines bridge the extremes:

1. **Guarded (no FS)**: same study-LOOCV splits and clean data as the guarded pipeline, but no feature selection—isolates the feature-selection effect.

2. **Naive clean**: same row-wise 5-fold split as the leaky pipeline, but with clean data and no feature selection—isolates the split-design effect from the leakage-feature effect.

All four workflows were audited with a permutation-gap test ($B = 500$), study-association testing, target scans, and duplicate detection.

7.2. Results

Table **??** summarizes the case-study output. The guarded workflow achieved an AUC of 0.610 with fold-to-fold standard deviation 0.063, whereas the leaky comparator achieved an AUC of 0.810 with standard deviation 0.024. The total AUC gap is therefore 0.200. The four-arm design decomposes this gap into three additive components:

1. **Feature-selection effect** (+0.072): guarded → guarded (no FS), both with study-LOOCV and clean data. Removing per-fold t-test feature selection raises the AUC from 0.610 to 0.682. This increase reflects the fact that **glmnet**'s built-in L_1 regularization selects relevant features more flexibly across all 2,000 genes than a rigid top-100 univariate filter.

2. **Split-design effect** (+0.035): guarded (no FS) → naive clean, both with clean data and no feature selection. Switching from study-LOOCV to row-wise 5-fold CV raises the AUC from 0.682 to 0.717. The increase represents optimistic bias: row-wise splitting allows samples from the same study to appear in both training and test sets, so study-specific expression patterns inflate the estimate.

Table 5: Case-study summary. The four columns form a decomposition chain: each successive pipeline changes exactly one factor (feature selection, split design, or leakage features), isolating its contribution to the AUC gap.

	Guarded	**Guarded (no FS)**	**Naive clean**	**Leaky**
Split design	Study LOOCV	Study LOOCV	5-fold CV	5-fold CV
Feature selection	t-test top 100	None	None	None
Leak features	No	No	No	Yes
AUC	0.610	0.682	0.717	0.810
SD	0.063	0.079	0.025	0.024
Perm. gap	0.101	0.173	0.208	0.301
Perm. p	0.002	0.002	0.002	0.002
Study-fold V	1.000†	1.000†	0.079‡	0.079
Multivar. scan p	0.2673	—§	—§	0.0099

$^{\dagger}V = 1$ is expected: each study-LOOCV fold is defined by study, so fold assignment and study identity are perfectly associated by design.
‡Same split as the leaky pipeline (dummy-subject 5-fold).
§Not computed for the sensitivity pipelines (audit limited to perm. gap).

3. **Leakage-feature effect** (+0.093): naive clean → leaky, both with row-wise 5-fold CV. Adding three leak-prone features raises the AUC from 0.717 to 0.810. Even with substantial noise ($\sigma = 1.0$), the outcome-derived feature provides the model with direct label information, demonstrating that noisy proxies of the target can cause substantial inflation.

The leakage features account for 47% of the raw AUC gap, feature selection for 36%, and the split-design change for 17%. Because the decomposition is sequential, these percentages are path-dependent: a different ordering of the three factors would redistribute the attributions. The chosen order (feature selection → split design → leakage features) moves from the most methodological choice to the most explicit data contamination, which we consider the most interpretable progression. The permutation gap is 0.101 (guarded), 0.173 (guarded, no FS), 0.208 (naive clean), and 0.301 (leaky). All four permutation null distributions center near 0.5 by construction (global label shuffle), and all p-values equal 0.002 (the floor at $B = 500$).

The study-fold association results deserve explicit interpretation. In the guarded workflow, Cramer's V is exactly 1.000 with a vanishingly small p-value because each outer fold is defined by study. This is not evidence of an implementation error; it is a property of the intended external-validation design. In the leaky comparator, the association is weak ($V = 0.079$, $p = 0.5302$) because the row-wise split ignores study boundaries. This contrast illustrates why association tables must be interpreted against the chosen resampling policy rather than in isolation.

The target-scan results illustrate the complementary value of multiple audit components (Figure **??**(a)). In the guarded workflow, no feature exceeded the conservative univariate threshold. The top association scores correspond to genes `XRCC4`, `XPA`, and `XPO1`, with scores 0.167, 0.143, and 0.097, respectively. In the leaky comparator, the noisy outcome feature `leak_global_y` had the highest score (0.540) but did *not* exceed the 0.9 flagging threshold. The weaker proxy features `leak_study_mean` and `leak_global_pc1` had scores 0.205 and 0.157. Despite the absence of univariate flags, the multivariate target scan distinguished the two workflows: the guarded analysis yielded $p = 0.2673$ (not significant at $\alpha = 0.05$),

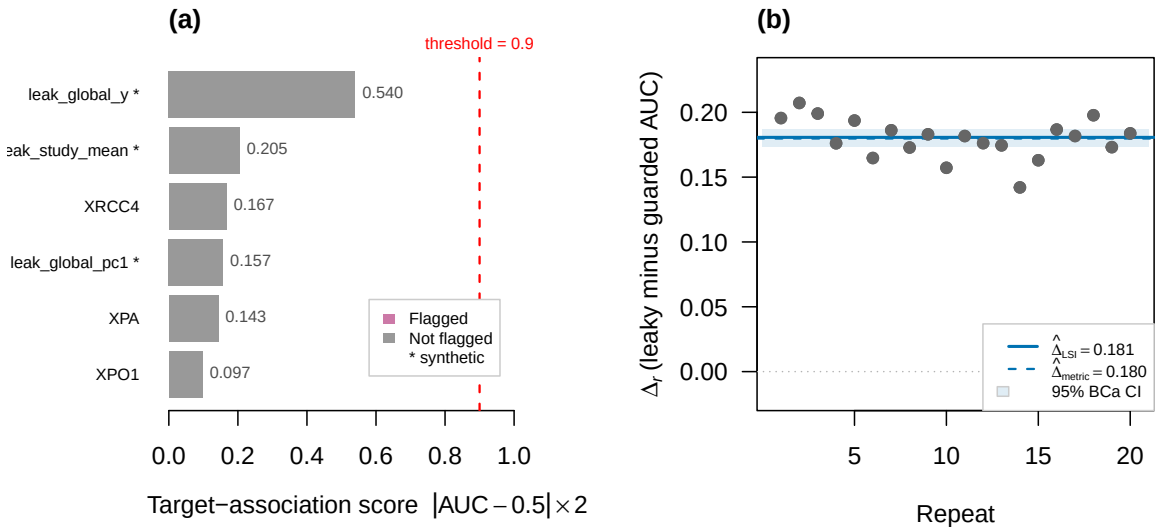

Figure 4: Case-study summaries. (a) Top univariate target-association scores (rescaled AUC, $|\text{AUC}-0.5|\times 2$) for the leaky pipeline feature matrix; asterisks mark injected synthetic leakage features. (b) Repeat-level inflation scores Δ_r (leaky minus guarded AUC) from `delta_lsi()` with 20 paired repeats; the solid line is the robust Huber estimate $\widehat{\Delta}_{\text{LSI}}$, the dashed line is the arithmetic mean, and the shaded band is the 95% BCa bootstrap interval.

whereas the leaky comparator yielded 0.0099. This pattern—a moderate univariate signal that escapes individual-feature flagging but is captured by the joint test—illustrates why the audit combines both scan types.

Duplicate detection returned a large number of unique near-duplicate sample pairs in both workflows: 21,435 pairs in the guarded workflow and 33,885 in the leaky workflow. Pairs were identified by cosine similarity on the reference expression matrix (threshold 0.995), canonicalized, and filtered to those that crossed training and test partitions in at least one fold (each pair counted once regardless of how many folds it spans). These likely correspond to technical replicates or near-identical expression profiles from the same patient across studies. The count is sensitive to the similarity threshold (here 0.995) and should be interpreted relative to the expected technical variability in the assay. This result is important for interpretation: leakage-aware study holdout alone does not guarantee independence when public data resources contain replicated or near-replicated samples across studies. In this case study, the study-LOOCV design already keeps same-study replicates on the same side of each fold, so the duplicates do not inflate the guarded AUC; however, in the naive 5-fold arms, near-identical samples from different studies can appear in both training and test sets, contributing to the observed performance gap. The duplicate screen therefore contributes information that is not captured by the resampling policy itself.

To quantify the AUC difference more formally, we applied `delta_lsi()` to the paired fits (both evaluated on the same batch-blocked fold structure with 20 repeats, as documented in `run_delta_lsi.R`; Figure **??**(b)). The arithmetic mean inflation was $\widehat{\Delta}_{\text{metric}} = 0.180$ AUC, and the robust Huber estimate was $\widehat{\Delta}_{\text{LSI}} = 0.181$ AUC, with a 95% BCa bootstrap

interval of [0.174, 0.187]. As Figure **??**(b) shows, all 20 repeat-level deltas are positive and tightly clustered between 0.14 and 0.21, with no outlying repeats—consistent with the near-identical robust and arithmetic estimates. The sign-flip randomization test yielded $p < 0.001$, supporting the conclusion that the naive pipeline exhibits systematic performance inflation relative to the guarded one. This pairing is important: because the naive and guarded pipelines share the same fold assignments, the repeat-level differences isolate the effect of leakage controls from the variance of fold sampling. The full `summary()` output for this `LeakDeltaLSI` object is shown in the code block in Section **??**. Note that the Δ_{LSI} captures the combined effect of all three factors; the four-arm decomposition in Table **??** is needed to separate their individual contributions.

Overall, the case study demonstrates the intended use of the package. A guarded workflow produces a more conservative cross-study estimate, while the audit components identify explicit target leakage and extensive near-duplicate overlap in the leaky comparator. No single audit component tells the full story: the permutation gap quantifies inflation, the multivariate target scan detects leakage features that escape univariate screening, and the duplicate detector reveals sample-level non-independence that persists even under study-blocked evaluation. The four-arm decomposition shows that nearly half (47%) of the AUC gap is attributable to explicit leakage features, while the remainder reflects methodological choices (feature selection and split design) that are not leakage per se but still affect the conservativeness of the estimate. The exercise also illustrates that some audit components, such as study-fold association, depend on the evaluation design and must be interpreted in context.

8. Software Design and Implementation

8.1. Object structure

The package uses S4 classes (`LeakSplits`, `LeakFit`, `LeakAudit`, `LeakDeltaLSI`; see Section **??**) to maintain explicit state across the workflow. Because fits and audits are self-contained objects, downstream summaries do not need to recompute the full model pipeline, and auditing can be decoupled from fitting—useful when model fitting is computationally expensive.

8.2. Pipeline components

Guarded preprocessing estimates all transformation parameters on the training fold and stores a transform function for applying them to the test fold. Supported operations include imputation (median, kNN, random-forest-based), normalization (z-score, robust scaling), variance and IQR filtering, and several univariate and model-based feature-selection methods. Character and factor levels are carried from training to test fold automatically.

Split construction encodes design decisions explicitly. Each mode uses the appropriate grouping or ordering column; `time_series` additionally supports prediction-horizon, purge, and embargo windows. A compact storage option reduces memory by storing a per-row fold vector rather than explicit index lists.

8.3. Reproducibility features

Several implementation details support reproducible analysis. Split objects store seeds and a content hash. Fit objects can retain sufficient information for refit-based permutation tests through `store_refit_data = TRUE`. Audit objects store both the parameters used and, optionally, the permutation draws themselves. This design is consistent with the principle that computational claims should remain inspectable at the level of code, data products, and derived summaries (**???**).

8.4. Reproducing the paper artifacts

All results reported in this manuscript are drawn from stored, seed-deterministic outputs. The main simulation comprises 9,000 tasks (180 configurations $\times$ 50 seeds), with a total sequential runtime of roughly three hours ($\sim$45 minutes on four cores). Because routine re-execution during review is impractical, results are distributed as stored `.rds` files in the replication materials.

Four self-contained generating scripts each set a deterministic seed and write per-task checkpoints. A compilation script (`compile_results.R`) derives every number in the manuscript from these stored objects, so a reader can audit the full chain from raw outputs to reported values. The checkpoint mechanism allows partial reruns of individual scripts. The stored artifacts total approximately 2 MB; per-task checkpoints add $\sim$35 MB. These materials are not included in the CRAN source tarball; they are distributed as replication files with this manuscript and in the GitHub repository.

8.5. Dependency structure

The core package has few hard dependencies. Optional imports extend functionality: **parsnip** and **recipes** for **tidymodels** interoperability, **glmnet** and **ranger** for common learners, **survival** for time-to-event metrics, and **rmarkdown** for HTML report rendering. The package remains usable for basic workflows even when some optional tools are not installed.

8.6. Relationship to existing software

bioLeak is not a general replacement for R machine-learning frameworks such as **caret**, **tidymodels**, or **mlr3**. Its role is narrower and complementary. Those frameworks provide broad model interfaces, preprocessing tools, and resampling utilities. **bioLeak** focuses specifically on leakage-aware split construction, guarded preprocessing, post hoc leakage diagnostics, and structured reporting. While **mlr3pipelines** (**??**) can encapsulate preprocessing within resampling, **bioLeak** adds a diagnostic layer—permutation-gap testing, target scans, duplicate detection, and structured reporting—that is not provided by general-purpose pipeline frameworks. The package can be used independently, but it is designed to coexist with existing modeling tools rather than to supersede them (**????**).

9. Discussion

Table 6: Function-level task-type coverage. ✓ = supported, (✓) = supported but not empirically evaluated in this paper, — = not yet implemented.

Function	Binary	Multiclass	Regression	Survival
`make_split_plan`	✓	✓	✓	✓
`fit_resample`	✓	✓	✓	✓
`tune_resample`	✓	✓	✓	—
`audit_leakage`	✓	(✓)	(✓)	(✓)
`delta_lsi`	✓	(✓)	(✓)	(✓)
Target scan	✓	(✓)	(✓)	—
`audit_report`	✓	(✓)	(✓)	(✓)

9.1. Software perspective

bioLeak addresses a practical gap in biomedical machine-learning workflows by pairing leakage-aware prevention mechanisms with post hoc diagnostic tools. The package design reflects a simple principle: performance estimation and leakage assessment should be treated as parts of one workflow, not as separate stages owned by different pieces of software. This integration is useful in biomedical settings because the causes of leakage are often spread across data preparation, resampling, and study design.

The package has several strengths. First, it makes the resampling geometry explicit. Users must choose whether the relevant independence unit is subject, batch, study, or time. Second, train-fold-only preprocessing is built into the central fitting interface. Third, the auditing framework creates a record of what was checked and how. Fourth, `delta_lsi()` offers a direct way to quantify how much reported performance changes when leakage controls are introduced. Taken together, these components support a style of model assessment that is more transparent than a single cross-validated AUC or accuracy estimate. In that sense, the package makes one practical response to the reproducibility concerns emphasized by **?**: leakage checks become part of the analysis record rather than an informal afterthought.

9.2. Limitations

Several limitations should be stated clearly. First, the package does not prove the absence of leakage. Audit components only test the evidence available in the stored predictions, metadata, and optional reference matrix. If a relevant grouping variable is omitted, no audit procedure can reconstruct it. Second, unsupervised learning workflows are not currently supported; the package is organized around supervised outcomes and supervised or outcome-linked audit procedures. Third, complex graph-based dependency modeling is not yet implemented. Current split construction is based on subject, batch, study, time, and related grouped constraints rather than on arbitrary dependency graphs.

The modeling interface supports binary classification, multiclass classification, continuous regression, and survival analysis, but function-level coverage varies by task type. Table **??** summarizes the current state. The audit diagnostics and simulation study presented here focus on binary classification. The permutation-gap test and `delta_lsi()` extend naturally to regression and survival metrics, but have not been evaluated empirically beyond classification in this manuscript.

Additional limitations are more specific to the current diagnostics. The univariate target scan is conservative by design and can miss indirect or multivariate proxies, while the stored

supplementary results show that the multivariate scan may respond to general predictive signal rather than to leakage alone. Duplicate detection depends on the supplied feature space and the chosen similarity threshold. Association tables can reflect the intended split policy rather than a defect, as the case study illustrates for study holdout. The `delta_lsi()` exchangeability options `by_group` and `within_batch` fall back to the iid engine with a warning (Section **??**); specialized randomization schemes for these structures are not yet implemented.

9.3. Future extensions

These restrictions are software design choices rather than conceptual impossibilities, and they define concrete directions for further development. Future work could expand the leakage taxonomy to include external-knowledge leakage, improve multivariate target-leakage calibration, extend nested tuning to survival models, and add more formal summaries for duplicate-risk severity. Another useful extension would be tighter integration with manuscript generation, so that audit reports, scalar summaries, and figure artifacts can be linked automatically in literate-analysis pipelines.

10. Summary

bioLeak provides an integrated R workflow for preventing and diagnosing data leakage in biomedical machine learning. The package pairs leakage-aware resampling and guarded preprocessing with post hoc permutation tests, batch-association diagnostics, target-leakage scans, and duplicate detection. The `delta_lsi()` framework offers formal inference for comparing naive and guarded pipelines. The simulation study shows that the permutation-gap test reliably detects several leakage mechanisms at moderate sample sizes, while the case study demonstrates how audit components interact on real multi-study transcriptomic data. Together, these tools make leakage assessment a documented, reproducible part of the modeling record.

11. Availability

As of April 2026, **bioLeak** is available on CRAN as version 0.3.5. The development source repository is hosted at https://github.com/selcukorkmaz/bioLeak. Package documentation includes a README, a long-form vignette (`bioLeak-intro`), and function-level help pages installed with the package. The replication materials accompanying this paper contain all generating scripts, stored result objects, and compilation code. The package is released under the MIT license.

Computational details

The results in this paper were obtained using R 4.5.2 (**?**). R itself and all packages used are available from the Comprehensive R Archive Network (CRAN) at https://CRAN.R-project.org/.

The package requires R $\geq$ 4.1.0 (for the native pipe operator `|>` used in some examples). The principal package is **bioLeak** 0.3.5. Key dependencies include **parsnip** for model specification,

recipes for preprocessing pipelines, **glmnet** for regularized regression, **pROC** for AUC computation, and **future** and **future.apply** for parallel execution. Exact version numbers for the analysis environment used in this paper are recorded in the `sessionInfo()` output included in the replication materials (`session_info.txt`).

References

Ambroise C, McLachlan GJ (2002). "Selection Bias in Gene Extraction on the Basis of Microarray Gene-Expression Data." *Proceedings of the National Academy of Sciences*, **99**(10), 6562–6566. doi:10.1073/pnas.102102699.

Arlot S, Celisse A (2010). "A Survey of Cross-Validation Procedures for Model Selection." *Statistics Surveys*, **4**, 40–79. doi:10.1214/09-SS054.

Bates S, Hastie T, Tibshirani R (2024). "Cross-Validation: What Does It Estimate and How Well Does It Do It?" *Journal of the American Statistical Association*, **119**(546), 1434–1445. doi:10.1080/01621459.2023.2197686.

Beam AL, Kohane IS (2018). "Big Data and Machine Learning in Health Care." *JAMA*, **319**(13), 1317–1318. doi:10.1001/jama.2017.18391.

Bengio Y, Grandvalet Y (2004). "No Unbiased Estimator of the Variance of K-Fold Cross-Validation." *Journal of Machine Learning Research*, **5**, 1089–1105.

Bischl B, Sonabend R, Kotthoff L, Lang M (2024). *Applied Machine Learning Using mlr3 in R*. CRC Press.

Cawley GC, Talbot NLC (2010). "On Over-Fitting in Model Selection and Subsequent Selection Bias in Performance Evaluation." *Journal of Machine Learning Research*, **11**, 2079–2107.

Christen P (2012). *Data Matching: Concepts and Techniques for Record Linkage, Entity Resolution, and Duplicate Detection*. Springer. doi:10.1007/978-3-642-31164-2.

Gentleman R, Lang DT (2007). "Statistical Analyses and Reproducible Research." *Journal of Computational and Graphical Statistics*, **16**(1), 1–23. doi:10.1198/106186007X178663.

Golub TR, Slonim DK, Tamayo P, Huard C, Gaasenbeek M, Mesirov JP, Coller H, Loh ML, Downing JR, Caligiuri MA, Bloomfield CD, Lander ES (1999). "Molecular Classification of Cancer: Class Discovery and Class Prediction by Gene Expression Monitoring." *Science*, **286**(5439), 531–537. doi:10.1126/science.286.5439.531.

Hamdan S, Love BC, von dem Hagen EAH (2023). "Confound-Leaking: Regression Leak in Machine Learning Applied to Neuroimaging." ArXiv preprint, 2306.12952.

Hastie T, Tibshirani R, Friedman J (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2nd edition. Springer. doi:10.1007/978-0-387-84858-7.

Huber PJ (1964). "Robust Estimation of a Location Parameter." *The Annals of Mathematical Statistics*, **35**(1), 73–101. doi:10.1214/aoms/1177703732.

Huber PJ, Ronchetti EM (2009). *Robust Statistics*. 2nd edition. Wiley. doi:10.1002/9780470434697.

Kapoor S, Narayanan A (2023). "Leakage and the Reproducibility Crisis in Machine-Learning-Based Science." *Patterns*, **4**(9), 100804. doi:10.1016/j.patter.2023.100804.

Kaufman S, Rosset S, Perlich C, Stitelman O (2012). "Leakage in Data Mining: Formulation, Detection, and Avoidance." *ACM Transactions on Knowledge Discovery from Data*, **6**(4), 1–21. doi:10.1145/2382577.2382579.

Kohavi R (1995). "A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection." In *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)*, volume 14, pp. 1137–1145.

Kuhn M (2008). "Building Predictive Models in R Using the **caret** Package." *Journal of Statistical Software*, **28**(5), 1–26. doi:10.18637/jss.v028.i05.

Kuhn M, Wickham H (2020). "Tidymodels: A Collection of Packages for Modeling and Machine Learning Using Tidyverse Principles." URL https://www.tidymodels.org.

Lang M, Binder M, Richter J, Schratz P, Pfisterer F, Coors S, Au Q, Casalicchio G, Kotthoff L, Bischl B (2019). "**mlr3**: A Modern Object-Oriented Machine Learning Framework in R." *Journal of Open Source Software*, **4**(44), 1903. doi:10.21105/joss.01903.

Leskovec J, Rajaraman A, Ullman JD (2014). *Mining of Massive Datasets*. 2nd edition. Cambridge University Press.

Moscovich A, Rosset S (2022). "On the Cross-Validation Bias Due to Unsupervised Pre-Processing." *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, **84**(4), 1474–1502. doi:10.1111/rssb.12537.

Peng RD (2011). "Reproducible Research in Computational Science." *Science*, **334**(6060), 1226–1227. doi:10.1126/science.1213847.

Phipson B, Smyth GK (2010). "Permutation P-values Should Never Be Zero: Calculating Exact P-values When Permutations Are Randomly Drawn." *Statistical Applications in Genetics and Molecular Biology*, **9**(1), Article 39. doi:10.2202/1544-6115.1585.

R Core Team (2025). *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria. URL https://www.R-project.org/.

Roberts DR, Bahn V, Ciuti S, Boyce MS, Elith J, Guillera-Arroita G, Hauenstein S, Lahoz-Monfort JJ, Schröder B, Thuiller W, Warton DI, Wintle BA, Hartig F, Dormann CF (2017). "Cross-Validation Strategies for Data with Temporal, Spatial, Hierarchical, or Phylogenetic Structure." *Ecography*, **40**(8), 913–929. doi:10.1111/ecog.02881.

Rosenblatt JD, Ben Simhon Y, Gal S, Gilron R (2024). "Practicalities of Resampling Methods for Brain Imaging: What Every Researcher Should Know." *NeuroImage*, **287**, 120520. doi:10.1016/j.neuroimage.2024.120520.

Sandve GK, Nekrutenko A, Taylor J, Hovig E (2013). "Ten Simple Rules for Reproducible Computational Research." *PLOS Computational Biology*, **9**(10), e1003285. doi:10.1371/journal.pcbi.1003285.

Sendak M, Elish MC, Gao M, Futoma J, Ratliff W, Nichols M, Bedoya A, Balu S, O'Brien C (2020). "The Human Body Is a Black Box: Supporting Clinical Decision-Making with Deep Learning." In *Proceedings of the Conference on Fairness, Accountability, and Transparency (FAT* '20)*, pp. 99–109. doi:10.1145/3351095.3372827.

Simon R, Radmacher MD, Dobbin K, McShane LM (2003). "Pitfalls in the Use of DNA Microarray Data for Diagnostic and Prognostic Classification." *Journal of the National Cancer Institute*, **95**(1), 14–18. doi:10.1093/jnci/95.1.14.

Soneson C, Delorenzi M (2014). "Batch Effect Confounding Leads to Strong Bias in Performance Estimates Obtained by Cross-Validation." *PLOS ONE*, **9**(6), e100335. doi:10.1371/journal.pone.0100335.

Stone M (1974). "Cross-Validatory Choice and Assessment of Statistical Predictions." *Journal of the Royal Statistical Society: Series B (Methodological)*, **36**(2), 111–147. doi:10.1111/j.2517-6161.1974.tb00994.x.

Subramanian J, Simon R (2013). "Overfitting in Prediction Models – Is It a Problem Only in High Dimensions?" *Contemporary Clinical Trials*, **36**(2), 636–641. doi:10.1016/j.cct.2013.06.011.

Tarca AL, Carey VJ, Chen Xw, Romero R, Drăghici S (2007). "Machine Learning and Its Applications to Biology." *PLOS Computational Biology*, **3**(6), e116. doi:10.1371/journal.pcbi.0030116.

Valavi R, Elith J, Lahoz-Monfort JJ, Guillera-Arroita G (2019). "**blockCV**: An R Package for Generating Spatially or Environmentally Separated Folds for k-Fold Cross-Validation of Species Distribution Models." *Methods in Ecology and Evolution*, **10**(2), 225–232. doi:10.1111/2041-210X.13107.

van de Mortel LA, van Wingen GA (2025). "Data Leakage Inflates Prediction Performance in Connectome-Based Machine Learning Models." *Nature Communications*, **16**, 1106. doi:10.1038/s41467-025-56082-4.

Varma S, Simon R (2006). "Bias in Error Estimation When Using Cross-Validation for Model Selection." *BMC Bioinformatics*, **7**, 91. doi:10.1186/1471-2105-7-91.

Wenger SJ, Olden JD (2012). "Assessing Transferability of Ecological Models: An Underappreciated Aspect of Statistical Validation." *Methods in Ecology and Evolution*, **3**(2), 260–267. doi:10.1111/j.2041-210X.2011.00170.x.

Winkler WE (1999). "The State of Record Linkage and Current Research Problems." *Statistical Research Report Series RR99/04*, U.S. Bureau of the Census.

Affiliation:

Selçuk Korkmaz
Department of Biostatistics and Medical Informatics
Faculty of Medicine
Trakya University
Balkan Yerleşkesi
22030 Edirne, Türkiye
E-mail: selcukorkmaz@gmail.com
URL: https://selcukorkmaz.github.io

Journal of Statistical Software http://www.jstatsoft.org/
published by the Foundation for Open Access Statistics http://www.foastat.org/
MMMMMM YYYY, Volume VV, Issue II *Submitted:* yyyy-mm-dd
doi:10.18637/jss.v000.i00 *Accepted:* yyyy-mm-dd